



ADR-013: TASCO Core as the Master for Products and Rating

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-013	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [Integration Architecture](#), [ADR-003 Rules engine and maker-checker](#), [Solution Architecture](#)

Context and problem statement

TASCO Insurance's core policy administration system holds the product catalogue, tariffs and rating, and issues every policy. The first version of the growth platform rated quotes locally from versioned rule sets (`config/rules/tariff*.json`, `rating*.json`), which was right for a sandbox but creates two sources of truth in production. A premium shown in the VETC app must be exactly the premium TASCO core binds and issues, and product or tariff changes must be made once, in core.

Decision drivers

- One source of truth for products and premiums; no re-keying.
- A customer must never pay a price that core has not confirmed.
- Core outages must not cause wrong prices; TASCO decides the trade-off between availability and certainty.
- Personal data sent to core is minimised.
- The sandbox and UAT must keep working without a core connection.

Considered options

1. **Two ports, `CoreRating` and `ProductCatalogue`, with a production REST adapter for TASCO core and a simulated core for sandbox, selected by `RATING_SOURCE`.**
2. Keep local rating and reconcile against core after issue.
3. Replicate core's tariff tables into the platform nightly and rate locally.

Decision outcome

Chosen option: 1.

Element	Decision
Ports	<code>CoreRating.quote()</code> returns per-line premium, VAT, total, a <code>coreQuoteRef</code> , <code>ratingVersion</code> and validity. <code>ProductCatalogue.fetchCatalogue()</code> returns products and versions.
Production adapter	<code>src/adapters/integrations/tascoCoreRatingClient.js</code> : REST over HTTPS (POST <code>/rating/v1/quotes</code> , GET <code>/products/v1/catalogue</code> ; paths to be confirmed against TASCO's specification), OAuth 2.0 client credentials with token caching, mTLS-ready, Idempotency-Key and X-Request-Id on every call, timeouts, retries and a circuit breaker, schema-validated responses, core error codes mapped to platform outcomes (declined, refer to underwriter, invalid)
Sandbox adapter	<code>src/adapters/integrations/simulatedTascoCore.js</code> : the same contract, priced from the approved local rule sets, returning core-style references
Orchestration	<code>src/application/ratingService.js</code>
Modes (<code>RATING_SOURCE</code>)	<code>rules</code> (local, default for development; refused in production unless <code>ALLOW_LOCAL_RATING=true</code>), <code>core</code> (fail closed: 503 when core is unavailable), <code>core_with_fallback</code> (an indicative local price is shown, marked indicative: <code>true</code> , and payment is blocked until POST <code>/api/quotes/:id/rerate</code> or POST <code>/api/customer/quotes/:id/rerate</code> obtains a core price)

Element	Decision
Binding	<code>purchase()</code> rejects indicative quotes and passes <code>coreQuoteRef</code> to <code>issuePolicy</code> , so core binds exactly the price it quoted
Data minimisation	Rating requests carry risk attributes only (vehicle category, usage, seats, age, holder type). Plate and holder name go to core only at issuance.
Catalogue	<code>catalogueService.sync()</code> (job <code>sync-catalogue</code> , CronJob 01:00 Vietnam time, or <code>POST /api/ops/jobs/catalogue-sync</code>) proposes a new <code>products</code> rule set and submits it for approval with the differences in the audit trail. It is never activated automatically, and the system actor cannot approve its own proposal.
Visibility	<code>GET /api/integrations/status</code> shows the rating source, circuit states and last catalogue sync

Consequences

- Good: core is the only authority on price, and the audit trail records which system rated each quote and with which version.
- Good: TASCO chooses availability (fallback) or certainty (fail closed) by configuration.
- Good: sandbox, UAT and automated tests run unchanged against the simulated core.
- Bad: the customer journey depends on core latency. A rating response time of 1 second or less at the 95th percentile must be agreed with TASCO core.
- Bad / constraints: the REST paths, payloads and error codes are assumptions until TASCO's interface specification is received in discovery; only the mapping inside the adapter changes. If core cannot expose a rating API in time, option 3 is the interim path, with core re-rating at issue and rejecting mismatches.

Pros and cons of the options

Reconcile after issue (option 2). Simple, but a mismatch is found only after the customer has paid. Rejected.

Replicate tariffs (option 3). Works without a rating API and is fast, but it duplicates rating logic and needs strict version control. Kept as the interim fallback path only.